Week 01 – Day 02 Assignment

Groq – Top 3 Models

1. LLaMA 3.3 70B

- **Context window:** 128K tokens
- **Max input tokens:** ~120K
- **Max output tokens:** ~8K
- **Strength:** Fast + strong general reasoning

---

2. GPT-OSS 120B

- **Context window:** 128K tokens
- **Max input tokens:** ~120K
- **Max output tokens:** ~8K
- **Strength:** Strong reasoning, open-weight alternative to GPT-4 class

---

3. Qwen3 32B

- **Context window:** 128K tokens
- **Max input tokens:** ~120K
- **Max output tokens:** ~8K
- **Strength:** Great for multilingual + structured tasks

Groq Pattern

- Most models standardized around:
  - **128K context**
  - **~8K output cap**
- Optimized for **speed over ultra-long context**

OpenRouter – Top 3 Models

1. Claude 3.5 Sonnet

- **Context window:** 200K tokens

- **Max input tokens:** ~190K
- **Max output tokens:** ~8K
- **Strength:** Best balance of reasoning + coding

---

2. GPT-4o

- **Context window:** 128K tokens
- **Max input tokens:** ~120K
- **Max output tokens:** ~16K
- **Strength:** Multimodal + strong general intelligence

---

3. DeepSeek R1

- **Context window:** 128K tokens
- **Max input tokens:** ~120K
- **Max output tokens:** ~32K
- **Strength:** Long reasoning chains + structured outputs

OpenRouter Pattern

- Wider variation:
  - Context: **128K → 200K+**
  - Output: **8K → 32K**
- Better for:
  - Long reasoning
  - Large document processing
  - Chain-of-thought workflows

Side-by-Side Comparison

Model	Context	Input	Output
LLaMA 3.3 70B	128K	~120K	~8K
GPT-OSS 120B	128K	~120K	~8K
Qwen3 32B	128K	~120K	~8K
Claude 3.5 Sonnet	200K	~190K	~8K
GPT-4o	128K	~120K	~16K
DeepSeek R1	128K	~120K	~32K

Key Takeaways

- **Groq models**
  - Uniform limits
  - Built for **speed + low latency agents**
  - Output size is the main constraint (~8K)
- **OpenRouter models**
  - Much more flexible
  - Better for:
    - Long outputs (DeepSeek R1)
    - Large context (Claude)

Practical Recommendation (for you)

Given you're exploring **Agentic AI**:

- Use:
  - **LLaMA 3.3 70B (Groq)** → fast tool execution
  - **Claude 3.5 Sonnet** → planning + reasoning
  - **DeepSeek R1** → deep reasoning chains

👉 This combo gives you:

- Speed + depth + flexibility

GENERATION OF TOKENS:

Tokens are the **fundamental units** LLMs actually "see" and generate—not words, not characters, but something in between. Understanding them properly gives you a big advantage when working with prompts, costs, and model behavior.

---

1. What is a token (really)?

A **token** is a chunk of text created by a tokenizer. It could be:

- A full word → `"apple"`
- Part of a word → `"un" + "believable"`
- A character → `"!"`
- Even whitespace → `" "`

Example:

```
"ChatGPT is amazing"
→ ["Chat", "G", "PT", " is", " amazing"]
```

👉 Models don't see meaning directly—they see **token IDs (numbers)**.

---

2. How tokens are generated (mechanism)

Step 1: Text → Tokenization

LLMs use algorithms like:

- **BPE (Byte Pair Encoding)**
- **Unigram LM (SentencePiece)**

Example (BPE idea)

Start with characters:

```
h e l l o
```

Merge common pairs:

```
he → hell → hello
```

Frequent patterns become **single tokens**.

👉 Result: Efficient compression of language into reusable chunks.

Step 2: Tokens → IDs

Each token maps to a number:

```
"hello" → 15496
"is" → 318
```

Now the model works with:

```
[15496, 318, ...]
```

Step 3: IDs → Embeddings

Each token ID becomes a vector:

```
15496 → [0.12, -0.98, 0.44, ...]
```

This vector captures:

- Semantic meaning
- Context relationships

Step 4: Prediction loop (core mechanism)

The model generates tokens **one at a time**:

```
Input: "The sky is"
→ Predict next token probabilities:

"blue" → 60%
"clear" → 20%
"dark" → 10%
```

Pick one → append → repeat.

👉 This is called **autoregressive generation**.

Step 5: Sampling (why outputs differ)

Instead of always picking the top token, models may:

- Use **temperature** (randomness)
- Use **top-k / top-p sampling**

This makes outputs:

- Creative (high randomness)
- Deterministic (low randomness)

---

3. Why tokens exist (motivation)

1. Efficiency (huge reason)

Language is repetitive.

Instead of:

```
c a t   → 3 units
```

Use:

```
"cat" → 1 token
```

👉 Fewer steps → faster training & inference.

---

2. Vocabulary compression

Instead of storing every word:

- Break into reusable subwords

Example:

```
"unhappiness"
→ "un" + "happi" + "ness"
```

👉 Model can understand new words it has never seen.

---

3. Handles all languages + symbols

Tokens can represent:

- English
- Code
- Emojis
- Mixed text

Example:

```
print("नमस्ते 🚀")
```

👉 Same tokenizer handles everything.

---

4. Enables generalization

Because tokens are reusable:

- Model learns patterns, not just words

Example:

```
"running", "runner", "runs"
→ share sub-tokens
```

👉 Better understanding of grammar & meaning.

---

4. Important behavior implications

1. Tokens ≠ words

Rule of thumb:

- 1 token ≈ 0.75 words (English)

But:

- Code → more tokens
- Indian languages → more tokens

---

2. Cost is token-based

APIs charge:

- Input tokens
- Output tokens

👉 Long prompts = higher cost

---

3. Context window = token limit

If model has 128K context:

- That includes:
  - Prompt

    - System instructions
    - Output

---

4. Tokenization affects performance

Bad prompt:

```
Explain this: [huge messy text]
```

Better:

- Structured
- Clean formatting

👉 Fewer tokens + better results

---

5. Mental model (simple analogy)

Think of tokens like:

🧩 **LEGO blocks for language**

- Words = structures
- Tokens = reusable pieces
- Model = predicts next block

---

6. Advanced insight (important for you)

Since you're exploring **Agentic AI**, tokens impact:

Latency

- More tokens → slower responses

Cost

- Agents with loops can explode token usage

Memory strategies

- Summarization

- Retrieval (RAG)
- Chunking

👉 All exist to **control token usage**

TL;DR

- Tokens = subword units
- Created via BPE/SentencePiece
- Converted to IDs → embeddings
- Model predicts next token step-by-step
- Designed for:
  - Efficiency
  - Generalization
  - Scalability